

Data Compression

Zeqian Duan

SJTU John Class

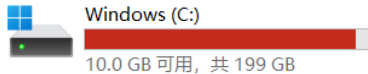
July 21, 2026

Outline

- ▶ Introduction
- ▶ Coding
- ▶ Lossy Compression
- ▶ Lossless Compression

Introduction

A tricky problem



Cannot send files larger than
1G.

OK

A tricky problem

Every day, the world generates 2.5 quintillion bytes of data.

A tricky problem

Every day, the world generates 2.5 quintillion bytes of data.
90% of global data was created in just the last two years.

A tricky problem

Every day, the world generates 2.5 quintillion bytes of data.
90% of global data was created in just the last two years.
More than 40% of internet data was generated by machines in 2020.

A tricky problem

Every day, the world generates 2.5 quintillion bytes of data.
90% of global data was created in just the last two years.
More than 40% of internet data was generated by machines in 2020.
By 2025, people will create 463 exabytes (EB) of data daily.

A tricky problem

Every day, the world generates 2.5 quintillion bytes of data.
90% of global data was created in just the last two years.
More than 40% of internet data was generated by machines in 2020.
By 2025, people will create 463 exabytes (EB) of data daily.
How can we deal with such huge amount of data?

Review

Sending images economically

Consider an image modelled by

$$A = \begin{bmatrix} a & a & c & c & e & e \\ a & a & c & c & e & e \\ a & a & c & c & e & e \\ a & a & c & c & e & e \\ a & a & c & c & e & e \\ a & a & c & c & e & e \end{bmatrix}$$

Observe that $\text{rank}(A) = 1$ and

$$A = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} \begin{bmatrix} a & a & c & c & e & e \end{bmatrix}.$$

Thus to send A which consists of 36 numbers, we only need to send out $6 + 6 = 12$ numbers.

Figure: Introduction of SVD

Coding

Why coding?

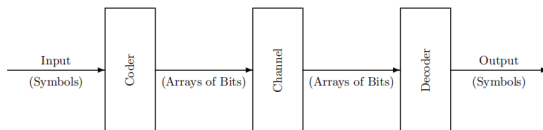


Figure: Generalized communication system

The core of data compression

Data compression is fundamentally an encoding and decoding process.

The core of data compression

Data compression is fundamentally an encoding and decoding process.

- ▶ Encoding: Transforms original data into a compact bit-stream (removes redundancy).

The core of data compression

Data compression is fundamentally an encoding and decoding process.

- ▶ Encoding: Transforms original data into a compact bit-stream (removes redundancy).
- ▶ Decoding: Reconstructs the data from the compressed stream.

The core of data compression

Data compression is fundamentally an encoding and decoding process.

- ▶ Encoding: Transforms original data into a compact bit-stream (removes redundancy).
- ▶ Decoding: Reconstructs the data from the compressed stream.

Key rule: Encoder and Decoder must share the same rules (dictionary/algorithm).

The core of data compression

Data compression is fundamentally an encoding and decoding process.

- ▶ Encoding: Transforms original data into a compact bit-stream (removes redundancy).
- ▶ Decoding: Reconstructs the data from the compressed stream.

Key rule: Encoder and Decoder must share the same rules (dictionary/algorithm).

Fixed-Length and Variable-Length Codes

Two fundamental ways to map symbols to bits:

Characteristics	FLC	VLC
Length	fixed	variable
Speed for decoding	Fast	Slow
Compression ratio	just so-so	more ideal
Robustness	strong	weak
Examples	ASCII	Morse

Table: Basic characteristics of mapping

Typical Examples

Control Characters				Digits			Uppercase			Lowercase		
HEX	DEC	CHR	Ctrl	HEX	DEC	CHR	HEX	DEC	CHR	HEX	DEC	CHR
00	0	NUL	^@	20	32	SP	40	64	@	60	96	`
01	1	SOH	^A	21	33	!	41	65	A	61	97	a
02	2	STX	^B	22	34	"	42	66	B	62	98	b
03	3	ETX	^C	23	35	#	43	67	C	63	99	c
04	4	EOT	^D	24	36	\$	44	68	D	64	100	d
05	5	ENQ	^E	25	37	%	45	69	E	65	101	e
06	6	ACK	^F	26	38	&	46	70	F	66	102	f
07	7	BEL	^G	27	39	'	47	71	G	67	103	g
08	8	BS	^H	28	40	(	48	72	H	68	104	h
09	9	HT	^I	29	41	)	49	73	I	69	105	i
0A	10	LF	^J	2A	42	*	4A	74	J	6A	106	j
0B	11	VT	^K	2B	43	+	4B	75	K	6B	107	k
0C	12	FF	^L	2C	44	,	4C	76	L	6C	108	l
0D	13	CR	^M	2D	45	-	4D	77	M	6D	109	m
0E	14	SO	^N	2E	46	.	4E	78	N	6E	110	n
0F	15	SI	^O	2F	47	/	4F	79	O	6F	111	o
10	16	DLE	^P	30	48	0	50	80	P	70	112	p
11	17	DC1	^Q	31	49	1	51	81	Q	71	113	q
12	18	DC2	^R	32	50	2	52	82	R	72	114	r
13	19	DC3	^S	33	51	3	53	83	S	73	115	s
14	20	DC4	^T	34	52	4	54	84	T	74	116	t
15	21	NAK	^U	35	53	5	55	85	U	75	117	u
16	22	SYN	^V	36	54	6	56	86	V	76	118	v
17	23	ETB	^W	37	55	7	57	87	W	77	119	w
18	24	CAN	^X	38	56	8	58	88	X	78	120	x
19	25	EM	^Y	39	57	9	59	89	Y	79	121	y
1A	26	SUB	^Z	3A	58	:	5A	90	Z	7A	122	z
1B	27	ESC	^[	3B	59	;	5B	91	[	7B	123	{
1C	28	FS	^\	3C	60	<	5C	92	\	7C	124	
1D	29	GS	]`	3D	61	=	5D	93	]	7D	125	}
1E	30	RS	^^	3E	62	>	5E	94	^	7E	126	~
1F	31	US	^^	3F	63	?	5F	95	_	7F	127	DEL

A	.-	K	--- --	U	--- --	0	-----	Question mark	..--..
B	--- --	L	--- --	V	--- --	1	-----	Apostrophe	--- --
C	--- --	M	--- --	W	--- --	2	-----	Parenthesis	--- --
D	--- --	N	--- --	X	--- --	3	-----	Quotation mark	..--..
E	---	O	--- --	Y	--- --	4	-----	Fraction bar	..--..
F	--- --	P	--- --	Z	--- --	5	-----	Equals	..--..
G	--- --	Q	--- --	Period	--- --	6	-----	Slash	..--..
H	--- --	R	--- --	Comma	--- --	7	-----	At sign	..--..
I	---	S	--- --	Hyphen	--- --	8	-----	Delete prior word	..--..
J	--- --	T	--- --	Colon	--- --	9	-----	End of Transmission	..--..

Figure: Morse Code

Figure: ASCII Character Set

A simple exercise to understand VLC

We need an efficient code suitable for everyday English text, one that is more efficient than ASCII (7 bits per character). The text we are interested in consists mainly of lowercase letters. In brief, we need to design a new way of coding to present 95 kinds of ASCII character and ETX, among which a ~ z and space (27 kinds in total) are more frequently used.

A simple exercise to understand VLC

We need an efficient code suitable for everyday English text, one that is more efficient than ASCII (7 bits per character). The text we are interested in consists mainly of lowercase letters.

In brief, we need to design a new way of coding to present 95 kinds of ASCII character and ETX, among which a ~ z and space (27 kinds in total) are more frequently used.

We observe that $2^5 = 32$, so is it a wise way to use merely 5 bits to present the 27 frequently-used characters?

Lossy Compression

Two types of Compression

- ▶ Lossless or reversible compression: can only be done if the original code was inefficient, for example by having unused bit patterns, or by not taking advantage of the fact that some symbols are used more frequently than others;
- ▶ Lossy or irreversible compression: the original symbol, or its coded representation, cannot be reconstructed from the smaller version exactly, but instead the expander produces an approximation that is “good enough”.

JPEG

JPEG is a kind of lossy compression, which is based on Discrete Cosine Transform (DCT).

JPEG

JPEG is a kind of lossy compression, which is based on Discrete Cosine Transform (DCT).

Note that DCT itself is a kind of lossless transform!

DCT

Consider the following formula:

$$F(u) = C(u) \sum_{x=0}^{N-1} f(x) \cos\left(\frac{(x+0.5)\pi}{N} u\right)$$

where $C(u)$ is a normalization factor.

DCT

Consider the following formula:

$$F(u) = C(u) \sum_{x=0}^{N-1} f(x) \cos\left(\frac{(x + 0.5)\pi}{N} u\right)$$

where $C(u)$ is a normalization factor.

Seem familiar?

DCT

Consider the following formula:

$$F(u) = C(u) \sum_{x=0}^{N-1} f(x) \cos\left(\frac{(x+0.5)\pi}{N} u\right)$$

where $C(u)$ is a normalization factor.

Seem familiar?

DCT is just a special case of Discrete Fourier Transform (DFT)!

DCT, cont'd

However, DCT can't be directly applied to every function like DFT does.

DCT, cont'd

However, DCT can't be directly applied to every function like DFT does.

The solution is also very simple: even extension.

DCT, cont'd

However, DCT can't be directly applied to every function like DFT does.

The solution is also very simple: even extension.

Given an array of real discrete signal $\{x[0], x[1], \dots, x[N-1]\}$, define x' as follows:

$$x'[m] = x[m], \quad \text{if } 0 \leq m \leq N-1$$

$$x'[m] = x[-m-1], \quad \text{if } -N \leq m \leq -1$$

DCT, cont'd

DCT is not used for compression itself, but it is used for energy concentration.

DCT, cont'd

DCT is not used for compression itself, but it is used for energy concentration.

If we just take the former n elements (something just like what we do after SVD), then we can compress the data as well as obtain more useful information.

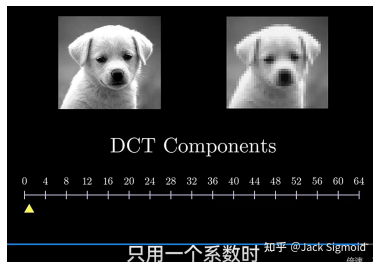
DCT, cont'd

DCT is not used for compression itself, but it is used for energy concentration.

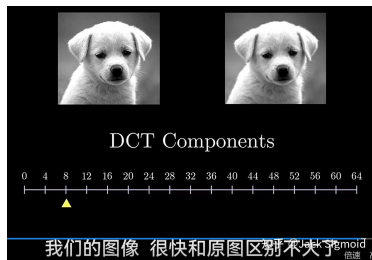
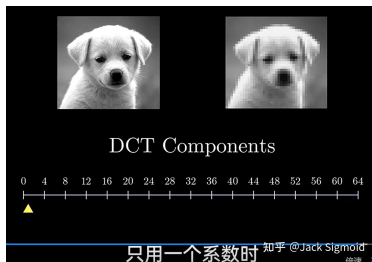
If we just take the former n elements (something just like what we do after SVD), then we can compress the data as well as obtain more useful information.

When we apply DCT to 2D version, that's exactly what JPEG does.

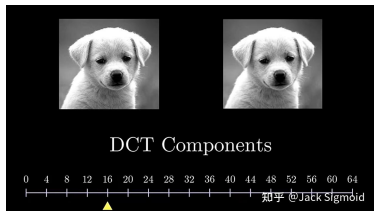
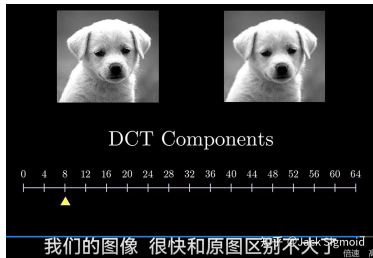
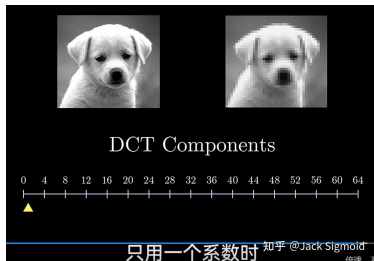
DCT, cont'd



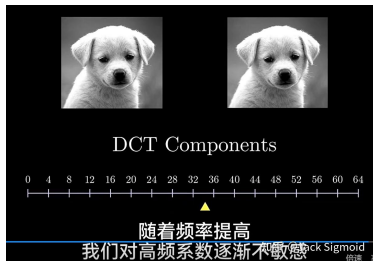
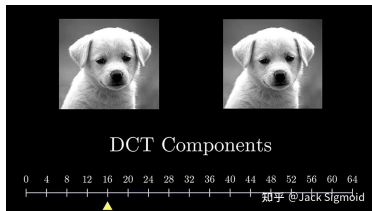
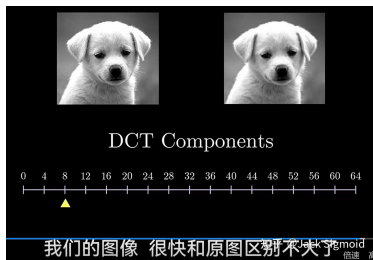
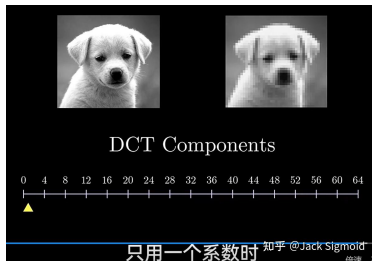
DCT, cont'd



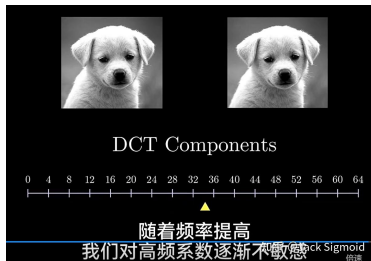
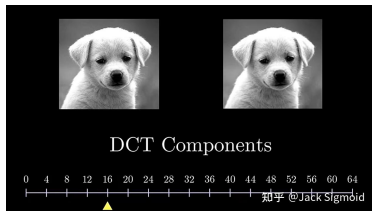
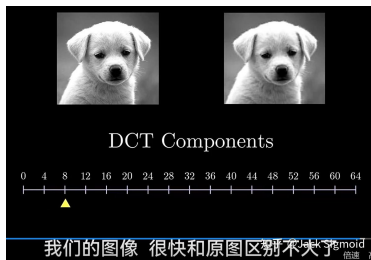
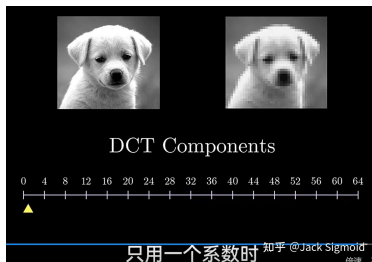
DCT, cont'd



DCT, cont'd



DCT, cont'd



<https://www.zhihu.com/question/39689253/answer/2490557388>

<https://www.youtube.com/watch?v=0me3guauqOU>

Lossless Compression

Lossless Compression

However, these methods can't avoid data loss, so it can't be applied to fields that require high accuracy. Thus, we have to consider another way to compress data.

Lossless Compression

However, these methods can't avoid data loss, so it can't be applied to fields that require high accuracy. Thus, we have to consider another way to compress data.

Usually, there are two ways of lossless compression. One is probability-based, another is dictionary-based.

Probability-based Lossless Compression

- ▶ Run Length Encoding (Not rigorous)

Probability-based Lossless Compression

- ▶ Run Length Encoding (Not rigorous)
- ▶ Arithmetic Coding

Probability-based Lossless Compression

- ▶ Run Length Encoding (Not rigorous)
- ▶ Arithmetic Coding
- ▶ Huffman Coding

Dictionary

If we don't use an algorithm (to count probability) to compress files, we usually need a dictionary to encode and decode.

Dictionary

If we don't use an algorithm (to count probability) to compress files, we usually need a dictionary to encode and decode.

There are three types of dictionaries:

Feature	Static	Semi-adaptive	Dynamic
Dictionary built	Before file	After analyzing file	During compression
Pre-scan needed	No	Yes (2 passes)	No
Dictionary transmitted	Yes	Yes (large)	No
Compression ratio	Poor	High	Medium-High
Decode speed	Fast	Fast (simple lookup)	Slower

A closer look at Dynamic Dictionary

Dynamic dictionary doesn't need to transmit the encoding (decoding) dictionary, since its dictionary is dynamically formed when the encoder (decoder) reads the data.

Since the rule is fixed, the dictionaries both sides form are the same. Thus, it dramatically relieve the stress of transmit.

A closer look at Dynamic Dictionary

Dynamic dictionary doesn't need to transmit the encoding (decoding) dictionary, since its dictionary is dynamically formed when the encoder (decoder) reads the data.

Since the rule is fixed, the dictionaries both sides form are the same. Thus, it dramatically relieve the stress of transmit.

A fantastic example: Lempel-Ziv-Welch Coding.

LZW Coding

Encoding			Transmission	Decoding	
Input	New dictionary entry		9-bit characters transmitted	New dictionary entry	Output
105 i	- -		256 (start)	- -	-
116 t	258 i t		105 i	- -	i
116 t	259 t t		116 t	258 i t	t
121 y	260 t y		116 t	259 t t	t
32 space	261 y space		121 y	260 t y	y
98 b	262 space b		32 space	261 y space	space
105 i	263 b i		98 b	262 space b	b
116 t	- -		- -	- -	-
116 t	264 i t t		258 i t	263 b i	i t
121 y	- -		- -	- -	-
32 space	265 t y space		260 t y	264 i t t	t y
98 b	- -		- -	- -	-
105 i	266 space-b i		262 space b	265 t y space	space b
116 t	- -		- -	- -	-
32 space	267 i t space		258 i t	266 space b i	i t
98 b	- -		- -	- -	-
105 i	- -		- -	- -	-
110 n	268 space b i n		266 space b i	267 i t space	space b i
-	- -		110 n	268 space b i n	n
-	- -		257 (stop)	- -	-

Table 3.2: LZW Example 1 Transmission Summary

Figure: an example of LZW

Thanks